

4.7 Hinge Loss

- Hàm mất mát Hinge: $\max(0, 1 - t)$
- Nó bằng 0 nếu $t \geq 1$ (điểm đã qua khỏi dải lề và nằm đúng phần).
- Nó tăng tuyến tính khi $t < 1$. Điều này tạo ra hình dạng cái bản lề.
- Hàm này không có đạo hàm tại $t = 1$, nhưng vẫn dùng Subgradient để tối ưu bằng SGD được.

4.8 Bài toán Đối ngẫu (Dual Problem)

- Bất kỳ bài toán Tối ưu hóa có ràng buộc (Primal Problem) nào cũng có một bài toán song sinh đi kèm gọi là bài toán Đối ngẫu (Dual Problem).
- Thường thì giải Dual dễ hơn Primal trong SVM khi số lượng quan sát m bé hơn số lượng đặc trưng n .
- Quan trọng hơn: Nhờ dạng Đối ngẫu mà phép màu **Kernel trick** mới có thể thực hiện được, điều mà dạng Primal không làm được.

4.9 Đẳng thức Kernel (Mercer's Theorem)

- Thay vì ánh xạ điểm x qua $\phi(x)$ (hàm biến đổi đa thức khổng lồ), ta chỉ cần tìm ra một hàm $K(\mathbf{a}, \mathbf{b})$.
- Hàm Kernel K tính ra **tích vô hướng** của $\phi(\mathbf{a})^T \phi(\mathbf{b})$ cực kì nhanh gọn.
- Các Kernel phổ biến:
 - Tuyến tính: $K(\mathbf{a}, \mathbf{b}) = \mathbf{a}^T \mathbf{b}$
 - Đa thức: $K(\mathbf{a}, \mathbf{b}) = (\gamma \mathbf{a}^T \mathbf{b} + r)^d$
 - Gaussian RBF: $K(\mathbf{a}, \mathbf{b}) = \exp(-\gamma \|\mathbf{a} - \mathbf{b}\|^2)$
 - Sigmoid: $K(\mathbf{a}, \mathbf{b}) = \tanh(\gamma \mathbf{a}^T \mathbf{b} + r)$

5.1 Ôn tập: Kiến trúc của SVM

- SVM là một mô hình phân loại dựa trên khoảng cách (Lề phân cách).
- Có hai biến thể là Tuyến tính (Dùng Hinge Loss, Solver cực nhanh) và Phi tuyến tính (Dùng Kernel).
- Lề cứng nghiêm ngặt tuyệt đối, lề mềm dùng tham số C để cân bằng lỗi.
- Các bài toán được mô hình hóa chặt chẽ dựa trên Quy hoạch Toàn phương.

5.2 Ứng dụng phổ biến của SVM

- Phân loại văn bản và phân tích cú pháp (Text/Categorization)
- Nhận dạng hình ảnh (Image recognition, MNIST)
- Phân loại gen tin sinh học (Bioinformatics, ung thư, protein)
- Hồi quy dự đoán tài chính.

5.3 Ưu điểm của SVM

- Hiệu quả với dữ liệu có chiều cao (Số lượng Đặc trưng lớn).
- Vẫn tối ưu cực tốt khi Không gian chiều dữ liệu lớn hơn cả số mẫu ($n > m$).
- Tiết kiệm bộ nhớ vì hàm ra quyết định chỉ phụ thuộc một nhóm các Support Vectors.
- Có nền tảng lý thuyết tối ưu lồi (Convex Optimization) bảo đảm hội tụ tới Tối ưu Toàn cục (Global Optimum), không bị dính vào tối ưu cục bộ như Neural Networks.

5.4 Nhược điểm của SVM

- Thời gian huấn luyện ($O(n^2)$ đến $O(n^3)$) rất chậm nếu bộ dữ liệu có trên 100,000 dòng.
- Không thể cho ra Xác suất (Probability) một cách tự nhiên như Logistic Regression (Phải dùng Platt Scaling tốn kém).
- Cực kỳ nhạy cảm và bắt buộc phải có bước chuẩn hóa (Scale) dữ liệu.
- Không diễn giải dễ dàng (Black-box model).

5.5 Bài tập Thực hành: Phân loại Ảnh Chữ Số

- Đề bài: Xây dựng bộ phân loại SVM trên bộ dữ liệu MNIST.
- MNIST gồm các ảnh xám 28x28 pixel của các chữ số viết tay từ 0-9.
- Vì SVM mặc định là nhị phân (Binary), nó sẽ sử dụng chiến lược One-versus-Rest (OvR) hoặc One-versus-One (OvO) tự động bên dưới Scikit-Learn.

5.6 Code Gợi ý Huấn luyện MNIST bằng SVM

```
from sklearn.datasets import fetch_openml
from sklearn.svm import SVC
from sklearn.metrics import accuracy_score

# 1. Tải dữ liệu và Scale (0-255 -> 0-1)
mnist = fetch_openml('mnist_784', version=1)
X, y = mnist["data"], mnist["target"]
X_train, X_test, y_train, y_test = X[:60000]/255., X[60000:]/255., y[:60000], y[60000:]

# 2. Xây dựng mô hình SVM với RBF Kernel (Mặc định)
svm_clf = SVC(gamma="scale")
svm_clf.fit(X_train[:10000], y_train[:10000]) # Huấn luyện tập nhỏ cho nhanh

# 3. Kiểm thử
y_pred = svm_clf.predict(X_test)
print("Accuracy:", accuracy_score(y_test, y_pred))
```

5.7 Lựa chọn Mô hình phù hợp

- Đừng lao vào Deep Learning ngay khi làm dự án thực tế.
- Luôn thử Logistic Regression hoặc Random Forest trước làm Baseline.
- Sau đó thử SVM. Dù tốc độ đào tạo chậm, nhưng khả năng phân loại của SVM thường vượt trội với tập dữ liệu tabular (bảng).

HỎI & ĐÁP